KOMOREBI / LANGUAGE AND COMPUTATION

OSL + Neuralese

Visual and Verification Guide

Komorebi P.

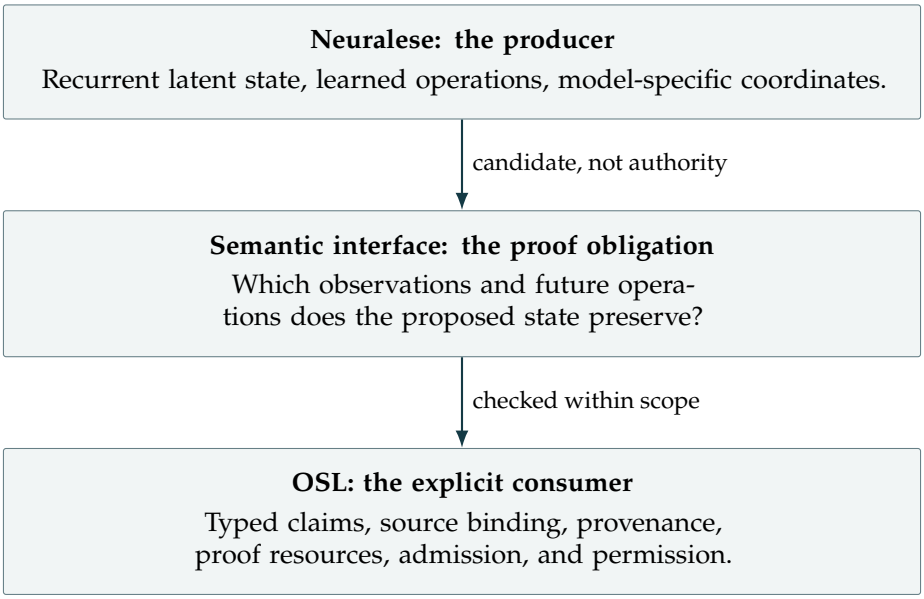
Consolidated research edition 1.0

6 October 2026

Two complementary projects, one explicit boundary. Neuralese studies learned recurrent computation. OSL specifies inspectable operations, provenance, and admission rules. The new reference interface joins a fully declared finite machine to the actual OSL runtime; it does not yet translate arbitrary learned neural states.

The manuscript, reference implementation, preserved source archives, and validation records form one edition. Publication finalization does not convert finite evidence into an unrestricted theorem or an unrun experiment into a result.

1 The architecture: two projects, not two names for one thing



What exists now. OSL-Core 0.5 is executable and has freshly replayed finite evidence. Neuralese contains learned finite operator results and their retained failures. The interface in this edition uses a complete, supplied finite transition table. Its checker is exact for that model.

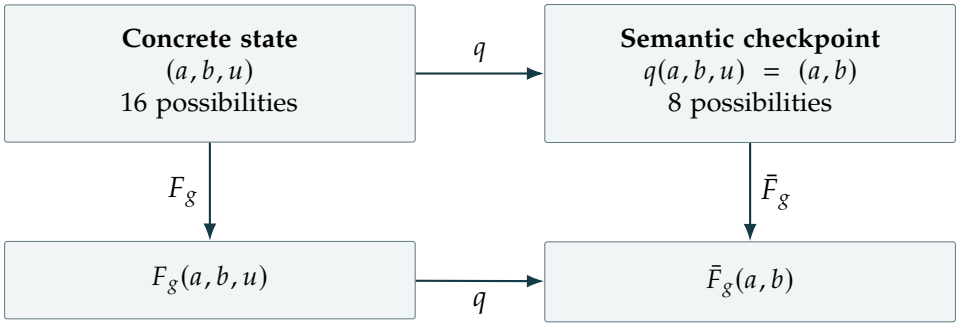
What remains a research obligation. A learned producer must yield a representation that the interface can interpret without borrowing the answer from an unrestricted translator. Checking a supplied table does not establish that a neural network implements the same table.

The connection is operational, not rhetorical. A source is accepted because stated equations or predicates pass, not because its description calls it "machine-native." The same system can accept one scope and retain another as unresolved.

Read next: OSL manuscript, Sections 1–3 and the semantic-checkpoint section; Neuralese manuscript, Sections 1–3. The proof-carrying and causal-abstraction background is cited in those papers.

2 A checkpoint that really preserves future computation

The new finite demonstrator has concrete states (a, b, u): eight visible D4 states (a, b) and a private bit u. It exposes sixteen concrete states but only eight semantic classes.



The square must commute at *every declared state*:

$$q(F_g(s)) = \bar{F}_g(q(s)), \qquad O(s) = \bar{O}(q(s)).$$

The operations are

$$r(a, b, u) = (a + 1, b, u), \quad f(a, b, u) = (-a, 1 - b, u), \quad v(a, b, u) = (a, b, 1 - u).$$

Since none of them makes the private bit visible, the checkpoint may discard it. The checker tests all states and generators. Induction then gives preservation for all finite operation words of this declared machine, not just a test-depth panel.

Checkpoint size is justified by future use. Losing u is permitted for this observation interface. Adding a new operation that copies u into the visible state would invalidate the old scope and require a new certificate.

The implementation is in `interface/semantic_interface.py`. It serializes the semantic class together with source, certificate, and scope digests. Those digests bind content; they do not authenticate a person or prove that the state arose in a particular historical episode.

3 Today's answer is not tomorrow's state

Consider a two-bit machine. Its current output is the first bit. Its next operation swaps the bits.

Concrete state	Output now	State after swap	Output then	Same present class?
$(0,0)$	0	$(0,0)$	0	Yes, with $(0,1)$
$(0,1)$	0	$(1,0)$	1	But future differs
$(1,0)$	1	$(0,1)$	0	Yes, with $(1,1)$
$(1,1)$	1	$(1,1)$	1	But future differs

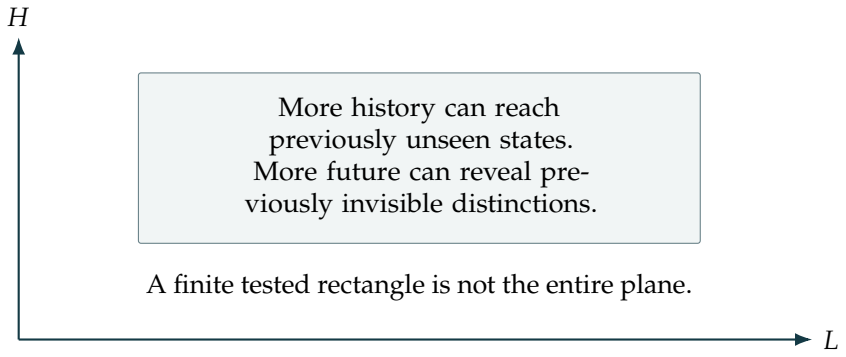
Present-only compression: retain one bit. There are two output classes.

Continuation-preserving compression: retain both bits. There are four classes because the next swap can reveal the discarded bit.

This is why a state that predicts an answer is not automatically a valid recurrent workspace. Neuralese needs both a history budget and a future-observation budget:

$$Q_{L,H} = R_{\leq L}/{\sim_H} .$$

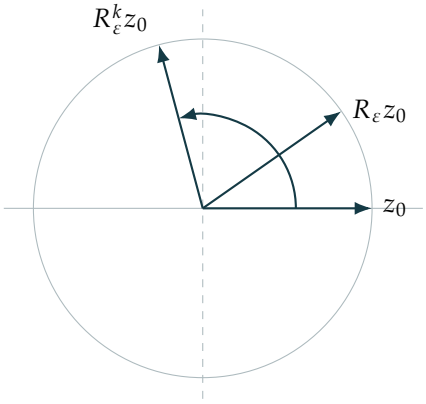
L says which states can have been reached. H says how many further operations can be used to distinguish them. These are independent axes.



The Neuralese completion-surface audit is organized around exactly this distinction. Many correct ordinary-history decisions coexist with delayed failures after longer paths. The figure is a definition diagram, not a measured heat map.

4 Numerical stability and semantic stability are different

A rotation preserves norm perfectly. It can still move a state into the wrong decision region.



All points have norm one.

The sign of the first coordinate changes.

In the explicit counterexample, the intended semantic action is the identity but the implementation is R_ε. Repetition accumulates an angular error $k\varepsilon$. A margin-based readout eventually changes class even though overflow, underflow, and norm explosion never occur.

What the later operator study contributes. It supplies an empirical counterpart to the distinction, not proof that every failure is a rotation. The hard-isometry architecture controls norm by construction; the overall learned-semantic gate conjunction still fails.

For normalized linear dynamics,

$$F_M(z) = \frac{Mz}{\|Mz\|}, \qquad DF_M(z) = \frac{(I - uu^\top)M}{\|Mz\|}.$$

The radial scale is removed, but the angular derivative need not contract. A claim about a stable semantic fiber must identify that fiber; a full-tangent derivative cannot silently substitute for it.

5 Read each result at its actual level

Record	What it supports, and what it does not
OSL-Core 0.5	168 fresh tests; corpus 66/66; 1,825 reproduced receipts. This is finite executable evidence, not universal interpreter soundness.
Stable Quotient	Strong two-seed structured-family and intervention results through tested depths. Arbitrary-family and longer-path limitations remain.
Operator Workspace	Completed 48-cell study. Eight of 26 primary gates pass. Norm preservation is not overall semantic success.
Completion Surface	More than 33 million correct ordinary-history continuation decisions. The registered cumulative and path-flatness failures remain.
Packet Closure	Training/execution complete, but assay A15 fails. The full developmental claim is not evaluated.
Invariant Kernel	Calibration stop. Untouched screen and pilot were not run.
Semantic Retraction	Armed, unspent, capacity-gated. 84 checkpoint-free tests are machinery checks, not learned results.
G1 pilot	Engineering passes, scientific conjunction fails. Shuffling retained the same IID accuracy.

Three questions must stay separate.

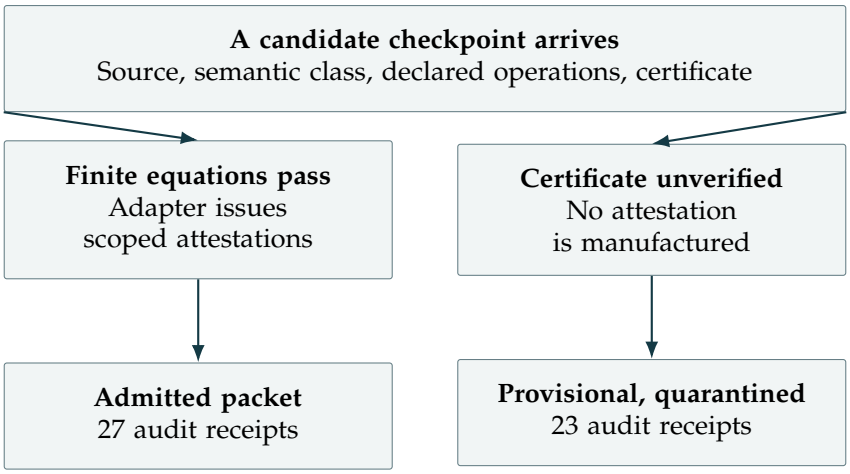
Did the code run? Did the assay measure the intended quantity? Did the substantive hypothesis pass its preregistered gates? A positive answer to the first is not a shortcut past the other two.

Fresh versus retained. This edition reruns bounded code and record checks. It does not retrain the historical neural models or declare their results independently replicated. The named archives preserve the original positive results, failures, and stop rules.

Source map: OSL manuscript's verification section; Neuralese manuscript's lineage and experimental sections; `SOURCE_LEDGER.json`; each project's `validation/` directory.

6 Admission is a transition with an obligation

The new demonstrator sends a finite checkpoint through the actual OSL-Core 0.5 runtime. Its two paths deliberately end differently.



Both paths pass the base receipt checker and the domain semantic checker. The quarantined path is a correct outcome, not a broken demonstration. Neither path invokes an external tool or performs a real-world action.

Provenance is not truth. An intact record can faithfully preserve a false input. A certificate can establish equations for a supplied finite model without proving that a learned model or the physical world realizes it. A digest commits to bytes, not to a principal's identity.

Similarly, communicating B's observation to A can justify A's use of communicated evidence without making it a firsthand observation by A. The distinction is represented in OSL's memory and provenance rules. A thousand copies of one observation should not become a thousand independent observations merely by passing through more agents.

Executable entry: `interface/run_demo.py`. The generated source, traces, final states, and checker outcomes are preserved under `interface/examples/`.

7 Reproduce the edition without consuming new experiments

From the complete package root:

```
python verify_release.py
python run_current_checks.py
python interface/run_demo.py --output ../local_demo
python build_manuscripts.py --output ../rebuilt_papers
```

Manifest verification checks the packaged bytes and recursive source-archive integrity. It is not a scientific efficacy certificate.

Current checks run the fresh finite interface, OSL-Core tests, corpus replay, and bounded Discovery Bridge checks in temporary working copies. Reports are written outside the sealed source tree. The Core 0.5 copy uses its original release directory name because its own manifest binds that name.

Manuscript rebuild requires a TeX distribution with the packages listed in the source. It writes to a separate destination and does not silently reseal the edition.

> **Do not run every discovered script indiscriminately.** The Semantic Retraction learned bank is unspent, the Entry–Cone learned endpoint is one-shot, and the invariant-kernel branch has a calibration stop. These rules remain in force. No default command in this edition opens a new learned bank, trains a model, or downloads a checkpoint.

Reading order. Start with the two new manuscripts. Consult the original 88-page OSL monograph for the broader historical mathematical development. Use the source ledger to select a Neuralese branch and its current results document before running its own commands. Source packages have separate environments and are not flattened into a fictional common neural runtime.

What finalization means here. The manuscripts, reference programs, reading map, claim status, and recoverable evidence are fixed together. The open learning and mechanization problems are stated rather than renamed. No Lean verification or general learned Neuralese-to-OSL translation is claimed.

Companion documents: *OSL: Operator-Symbolic Language and Proof-Carrying Interfaces*; *Neuralese: Recurrent Latent Operators, Causal Semantics, and Stability*. Research edition 1.0, Komorebi P., 6 October 2026. The full scholarly bibliography and version-bound internal references are in the manuscripts.